PRACTICAL NO. 7

AIM: A computer system has four different types of resources (printer, ROM, hard disk and RAM) and it needs to complete execution of five processes (Google Drive, Firefox, Word Processor, Excel and PowerPoint) The number of allocated resources, maximum needed resources to complete the execution and total available resources should be taken from the user. (Sample data for reference is given below)

Process	Allocation Printer	Allocation ROM	Allocation Hard Disk	Allocation RAM	Max Printer	Max ROM	Max Hard Disk	Max RAM
Google Drive	0	0	1	4	0	6	5	6
Firefox	0	6	3	2	0	6	5	2
Word Processor	0	0	1	2	0	0	1	2
Excel	1	0	0	0	1	7	5	0
PowerPoin t	1	3	5	4	2	3	5	6

Available Resources			
Printer	**ROM**	**Hard Disk**	**RAM**
1	6	2	0

Write a program to:

- Calculate current need of resources by each process Find whether these processes can be executed with available resources. If yes, show the correct order of process execution.

CODE:

MINGW64:/c/Users/Haris/Desktop

GNU nano 8.7 bankers.c

```c
#include <stdio.h>

#define MAXP 10
#define MAXR 10

int main() {

    int p, r;
    int allocation[MAXP][MAXR], max[MAXP][MAXR];
    int need[MAXP][MAXR];
    int available[MAXR];
    int finish[MAXP] = {0};
    int safeSeq[MAXP];

    int i, j, k, count = 0;

    printf("Enter number of Processes: ");
    scanf("%d", &p);

    printf("Enter number of Resources: ");
    scanf("%d", &r);

    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < p; i++) {
        for(j = 0; j < r; j++) {
            scanf("%d", &allocation[i][j]);
        }
    }

    printf("\nEnter Max Matrix:\n");
    for(i = 0; i < p; i++) {
        for(j = 0; j < r; j++) {
            scanf("%d", &max[i][j]);
        }
    }

    printf("\nEnter Available Resources:\n");
    for(i = 0; i < r; i++) {
        scanf("%d", &available[i]);
    }

    // Calculate Need Matrix
    for(i = 0; i < p; i++) {
        for(j = 0; j < r; j++) {
            need[i][j] = max[i][j] - allocation[i][j];
        }
    }

    // Display Allocation Table
    printf("\n---------------- Allocation Matrix ----------------\n");
    for(i = 0; i < p; i++) {
        printf("P%d\t", i);
        for(j = 0; j < r; j++) {
            printf("%d ", allocation[i][j]);
        }
        printf("\n");
    }
```

MINGW64:/c/Users/Haris/Desktop

GNU nano 8.7 bankers.c

```c
    // Display Max Table
    printf("\n---------------- Max Matrix ----------------\n");
    for(i = 0; i < p; i++) {
        printf("P%d\t", i);
        for(j = 0; j < r; j++) {
            printf("%d ", max[i][j]);
        }
        printf("\n");
    }

    // Display Need Table
    printf("\n---------------- Need Matrix ----------------\n");
    for(i = 0; i < p; i++) {
        printf("P%d\t", i);
        for(j = 0; j < r; j++) {
            printf("%d ", need[i][j]);
        }
        printf("\n");
    }

    printf("\nAvailable Resources:\n");
    for(i = 0; i < r; i++) {
        printf("%d ", available[i]);
    }
    printf("\n");

    // Safety Algorithm
    while(count < p) {
        int found = 0;

        for(i = 0; i < p; i++) {
            if(finish[i] == 0) {

                for(j = 0; j < r; j++) {
                    if(need[i][j] > available[j])
                        break;
                }

                if(j == r) {
                    for(k = 0; k < r; k++)
                        available[k] += allocation[i][k];

                    safeSeq[count++] = i;
                    finish[i] = 1;
                    found = 1;
                }
            }
        }

        if(found == 0) {
            printf("\nSystem is NOT in Safe State!\n");
            return 0;
        }
    }

    printf("\nSystem is in SAFE State.\nSafe Sequence: ");
    for(i = 0; i < p; i++) {
        printf("P%d ", safeSeq[i]);
    }

    printf("\n");
    return 0;
```

OUTPUT:

```
Haris@LAPTOP-65NO3OIR MINGW64 ~/Desktop
$ nano bankers.c

Haris@LAPTOP-65NO3OIR MINGW64 ~/Desktop
$ gcc bankers.c -o bankers

Haris@LAPTOP-65NO3OIR MINGW64 ~/Desktop
$ ./bankers
Enter number of Processes: 5
Enter number of Resources: 4

Enter Allocation Matrix:
0 0 1 4
0 6 3 2
0 0 1 2
1 0 0 0
1 3 5 4

Enter Max Matrix:
0 6 5 6
0 6 5 2
0 0 1 2
1 7 5 0
2 3 5 6

Enter Available Resources:
1 6 2 0

---------------- Allocation Matrix ----------------
P0      0 0 1 4
P1      0 6 3 2
P2      0 0 1 2
P3      1 0 0 0
P4      1 3 5 4

---------------- Max Matrix ----------------
P0      0 6 5 6
P1      0 6 5 2
P2      0 0 1 2
P3      1 7 5 0
P4      2 3 5 6

---------------- Need Matrix ----------------
P0      0 6 4 2
P1      0 0 2 0
P2      0 0 0 0
P3      0 7 5 0
P4      1 0 0 2

Available Resources:
1 6 2 0

System is in SAFE State.
Safe Sequence: P1 P2 P3 P4 P0
```